

Technical Documentation

AI-Agent (Auto part Storage)



Groepsnaam: Fatneek

Docent: Dhr. Christian MS CS/ Mevr. Bergwijn

Sem: 5.2

Groepsleden:

- Ramkhelawan Riaaz
- Sodipo Sherreskly
- Sangham Rishika
- Ramdhiansing Shakeel
- Sariman Jalen

Inhoudsopgave

Waarom dit project:	3
TopGear:	3
Het probleem:	3
Ons oplossing met AI:	4
Functies:	4
1. Systeemarchitectuur & design (High-level)	5
1.1 Architectuur diagram:	5
1.2 Tech Stack Rechtvaardiging:	6
2. Data Architectuur	8
2.1 Overzicht:	8
2.2 Opbouw Data Model (ERD):	8
2.3 Secure Data Quality:	9
2.4 Data Flow	11
3. AI & LLM Integration	12
3.1 Keuze van het LLM Platform (Groq):	12
3.2 Promptstructuur (System, Developer en User Prompt):	12
3.3 JSON-gebaseerde output:	14
3.4 Function calling:	15
4. Infrastructuur & deployment	16
5. Onderhoud en betrouwbaarheid	18
6. Schaalbaarheid van het systeem	20

Lijst met Figuren

Figuur 1: Architectuurdiagram	5
Figuur 2: Database ERD	8
Figuur 3: Sequence Diagram	11
Figuur 4: Vb. System Prompt	13
Figuur 5: Vb. User Prompt	13
Figuur 6: Vb. JSON output	14

Waarom dit project:

Het is een veel gezegde uitspraak, dat als je niet meegaat met de ontwikkeling van technologie dan kan je veel kansen missen, vooral als ondernemer. Tegenwoordig is AI the hot topic niet alleen binnen het IT-wereld, maar bijna over alle sectoren. Het is er niet alleen om leven makkelijker te maken maar om te helpen bij het uitvinden van nieuwe technieken en nog meer.

Wij als team demonstreren daarom binnen dit project hoe AI geïmplementeerd kan worden binnen een bepaalde situatie, waarbij wij zagen dat er een probleem was en AI hierbij een verbetering kan brengen. Hieronder vertellen wij globaal over de case van ons project.

TopGear:

TopGear is een auto-onderdelenwinkel (autoparts shop) die auto-onderdelen en accessoires verkoopt, zoals remblokken, oliefilters, accu's en nog veel meer. De organisatie ondersteunt klanten bij het vinden van het juiste onderdeel dat past bij hun voertuig. Zo een winkel/ shop wordt bezocht door:

- Particuliere autobezitters
- Garages/ monteurs
- Schadeherstel & spuiterijen
- Bedrijven met wagenpark (taxi, koerier, transport, verhuur)
- Auto-enthousiastelingen/ tuners

Het probleem:

Problemen die deze bezoekers vaak hebben, wanneer ze een auto part shop naar binnen lopen:

- Ze weten niet precies **welk onderdeel** ze nodig hebben (alleen klacht: "hij piept/trekt/lekkage").
- Ze hebben **verkeerde voertuiginfo** (bouwjaar, motorvariant, chassis/VIN) → onderdeel past niet.
- Ze brengen het **oude onderdeel niet mee** en kunnen het niet vergelijken.
- **Onderdeel-namen** worden door elkaar gehaald (remschijf vs. remblok, bougie vs. bobine, filtertypes).
- **Retour/garantie** gedoe na verkeerd kopen of foutief monteren.

Kort gezegd: Voor de bezoeker leidt dit vaak tot **verkeerde aankopen, extra kosten, vertraging en stress** doordat het juiste onderdeel niet wordt gevonden/past of niet op tijd beschikbaar is en montage onzeker is. Voor de shop betekent het **meer retourneren/garantiegevallen, langere balietijd, hogere operationele kosten en omzetverlies**, plus risico op **ontevreden klanten en reputatieschade**.

Ons oplossing met AI:

Een AI-agent op de website van de shop. De AI-agent wordt een functionaliteit die op de website van de shop komt te staan, waarmee de bezoekers/ gebruikers makkelijk het juiste onderdeel kunnen vinden en ook een snelle afhandeling kunnen ondergaan.

Functies:

- De AI-agent kan voor de gebruiker makkelijk achterhalen of de autopart shop, wel de juiste onderdelen heeft, door gebruik te maken van product nummer of overige informatie. (Search)
- De AI-agent kan voor de gebruiker een reservering doen op het product/ component. (Make reservation on product)
 - o Restriction1: Reservering kan alleen max 4-6 uren geldig zijn, om misbruik van het reservering systeem te voorkomen.
 - o Restrction2: Gebruiker kan max 3-4 items reserveren per account.
- De AI-agent zal de gebruiker voorzien van: prijzen, informatie, etc. over producten.
- De AI-agent kan gewoon gebruikt worden voor normale interacties.

In de overige hoofdstukken, word dit product technisch uitgewerkt, met de bedoeling dat de lezer begrip kan krijgen over de opbouw van het product.

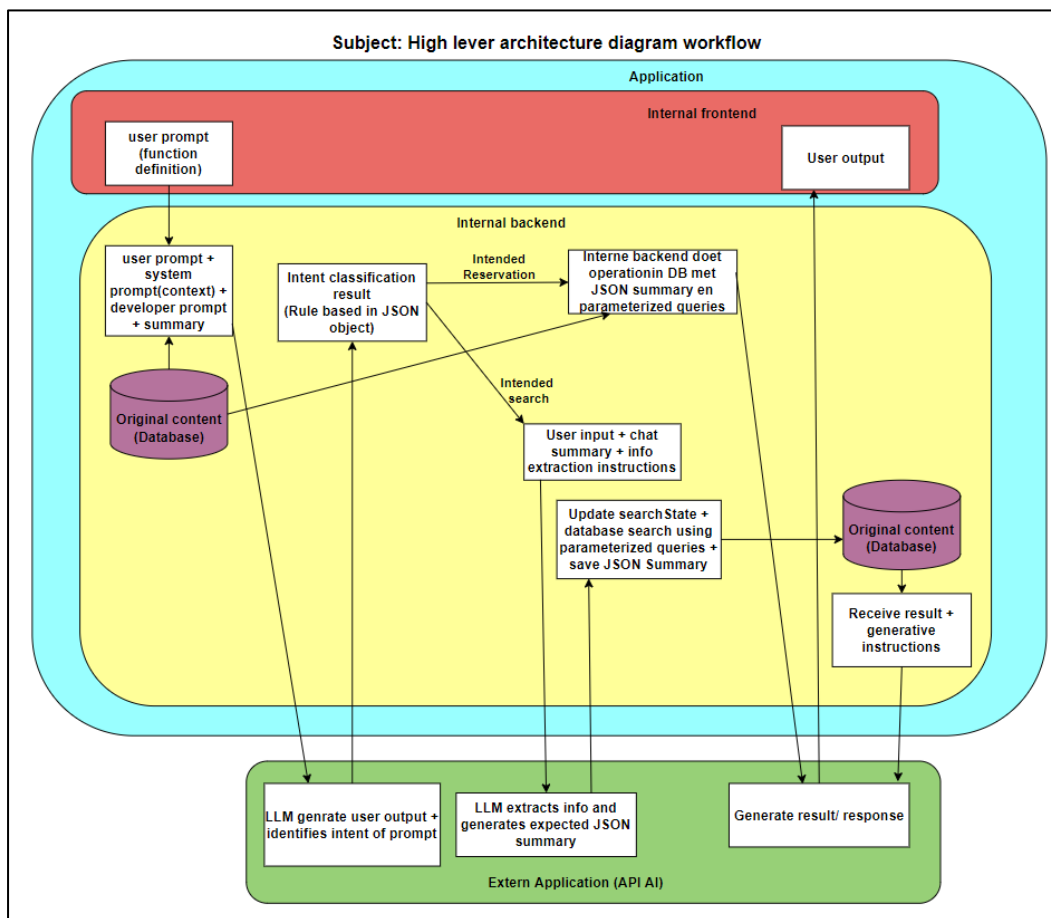
1. Systeemarchitectuur & design (High-level)

De focus ligt op het digitaliseren van het afhandelingsproces tussen klant en autopart shop via een webplatform met twee kanten: een klantportaal voor het opzoeken en aanvragen van onderdelen, en een medewerker interface om voorraad en klantverzoeken te beheren. Maar de **chatbot** vormt hierbij het kernonderdeel en is de belangrijkste schakel in de communicatie tussen klant en systeem.

1.1 Architectuur diagram:

Hieronder vindt u de opgestelde architectuur van de chatbot. Deze architectuur laat duidelijk zien hoe de chatbot functioneert binnen de drie belangrijkste lagen van het systeem. Het is belangrijk om deze architectuur vast te leggen, omdat het de bouwers van het systeem richting geeft in wat er gebouwd moet worden en op welke manier.

Het systeem bestaat uit een **frontendlaag** voor gebruikers/klanten, een **backendlaag** waarin alle interne verwerkingen plaatsvinden op basis van gebruikersverzoeken, en een **externe AI-laag** omdat er gebruik wordt gemaakt van een externe AI-platform via een API (gemaakte keuzes worden verder uitgewerkt). Binnen de workflow is er ook bewust keuzes gemaakt om misbruik te beperken en de beveiliging van gevoelige data te waarborgen.



Figuur 1: Architectuurdiagram

Voor de gekozen situatie is er bewust keuze gemaakt om van een paar bekende technieken te gebruiken. Deze technieken zijn:

- **User + system + developer prompt:** De LLM ontvangt een combinatie van user-, system- en developer-prompts, waarbij de system prompt het algemene gedrag van het model bepaalt, de developer prompt specifieke instructies voor de applicatie bevat en de user prompt de daadwerkelijke vraag van de gebruiker levert.
- **Intent classification:** Intent classification wordt gebruikt om automatisch te bepalen welke actie de chatbot moet uitvoeren (bijvoorbeeld SEARCH of RESERVATION) door de gebruikersinput te analyseren met een LLM.
- **JSON-objecten opstellen:** De chatbot structureert de geïnterpreteerde informatie uit de gebruikersinput in JSON-objecten, zodat de backend deze data consistent kan verwerken en opslaan in de chat summary.
- **Function call:** Function calling wordt toegepast om, op basis van de gedetecteerde intent, specifieke backend-functionaliteiten zoals database searches of reserveringen aan te roepen.
- **Parameterized queries:** Parameterized queries worden gebruikt bij database-interacties waarbij invoerwaarden via parameters aan SQL-queries worden meegegeven, zodat SQL-injection wordt voorkomen en queries veilig uitgevoerd kunnen worden.

Opm.: Deze architectuur is een high-level architectuur van de interactie tussen gebruiker en de AI-agent, dus vandaar onderdelen zoals security en port nummer niet aangehaald worden, maar word wel geïmplementeerd in het systeem.

1.2 Tech Stack Rechtvaardiging:

1. Front-end:

- Framework: Next.js 16 + React
- Deployment: Vercel
- Technologieën: TypeScript, CSS, React Components

Building workflow frontend:

1. Ontwikkelen in VSCode.
2. Dependencies installeren via npm install.
3. Lokaal testen en debuggen (npm run dev).
4. Feature-branch pushen naar GitHub.
5. Merge naar main → Vercel buildt en deployed frontend + backend.
6. Preview deployments via Vercel voor UI-controle.

2. Back-end:

- Framework: Next.js 16.
- Database: Neon (serverless PostgreSQL).
- Authenticatie: NextAuth.js.

- Deployment: Vercel.

Workflow backend:

1. Code schrijven in VSCode.
2. Lokaal testen via npm run dev (Next.js development server).
3. Feature-branch maken in GitHub en code pushen.
4. Code review uitvoeren, daarna merge naar main branch.
5. Vercel detecteert merge → bouwt en deployt automatisch de nieuwe productieversie (zonder downtime).

3. Database:

- Database Management System: PostgreSQL.
- Cloud Database Platform: Neon (Serverless PostgreSQL).

De applicatie maakt gebruik van een **Relationeel Database Management Systeem (RDBMS)**.

Voor de implementatie is gekozen voor **PostgreSQL**, een open-source relationele database die bekend staat om zijn betrouwbaarheid, schaalbaarheid en ondersteuning voor complexe SQL-query's.

De database wordt gehost via **Neon**, een cloudplatform dat **serverless PostgreSQL databases** aanbiedt. Neon biedt onder andere:

- Automatische schaalbaarheid
- Snelle provisioning van database-instances
- Database branching voor ontwikkel- en testomgevingen
- Veilige rollbackmogelijkheden bij fouten

Hierdoor kunnen wijzigingen eerst getest worden in een **ontwikkel- of testomgeving**, voordat deze in productie worden toegepast. Dit maakt Neon een **flexibele, schaalbare en betrouwbare oplossing** voor het beheer van de database van de applicatie.

4. AI/ chatbot Componenten:

Over dit onderdeel vindt u meer in hoofdstuk 3:

- Large Language Model (LLM)
- Intent Classification
- Structured JSON-state management
- Prompt Engineering (system + developer + user prompts)
- Function-based routing (search/ reservation handlers)

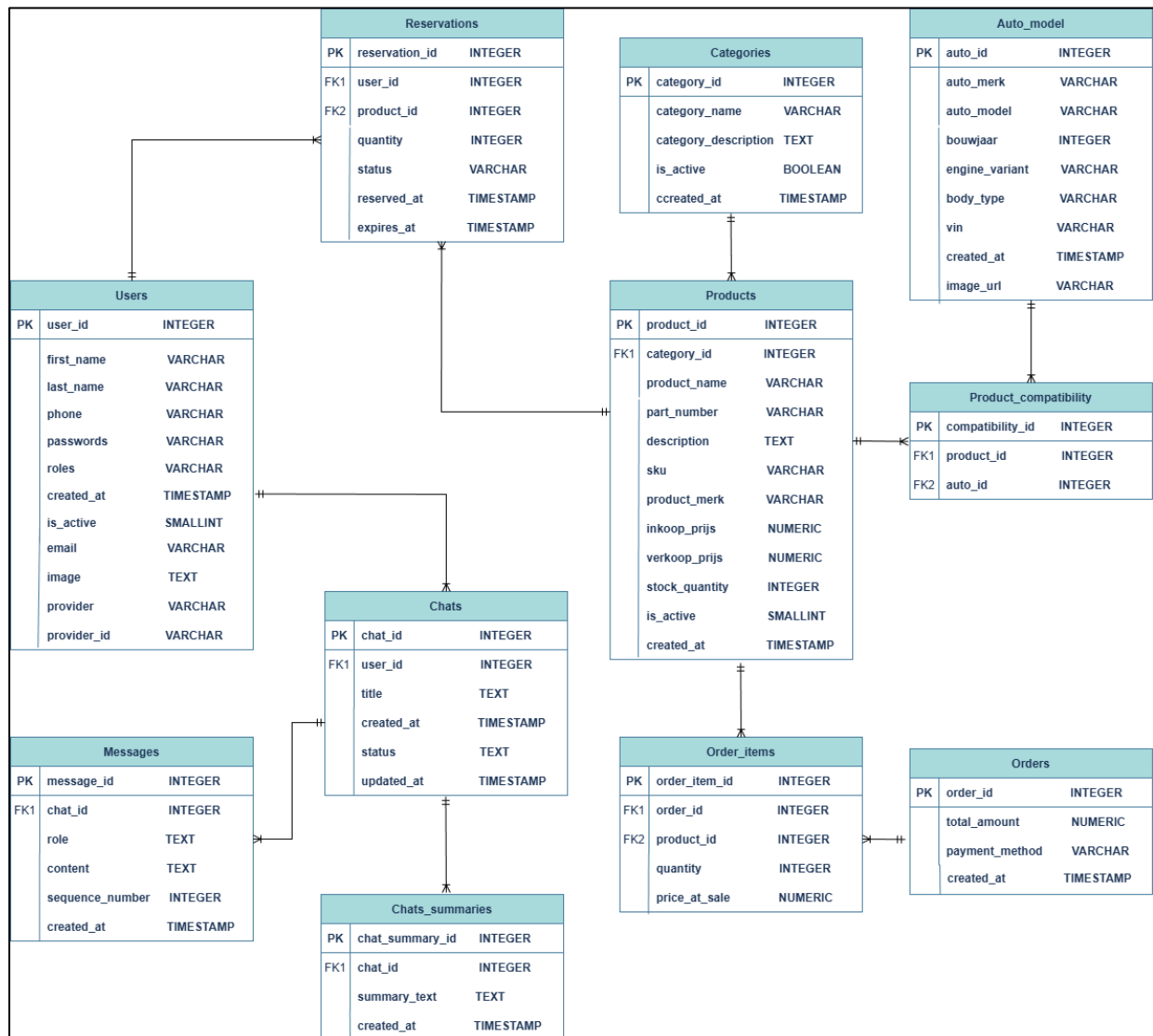
2. Data Architectuur

Dit hoofdstuk beschrijft hoe data binnen de applicatie worden beheerd, van verzameling en transformatie tot distributie en gebruik. Het vormt de blauwdruk voor de datastroom binnen het systeem. Dit onderdeel is van groot belang, *omdat het systeem ontworpen moet worden om informatie snel op te halen en efficiënt te verwerken*, waardoor hier extra aandacht en tijd aan wordt besteed.

2.1 Overzicht:

De data-architectuur van het systeem is gebaseerd op een **relationeel datamodel** waarbij data wordt opgeslagen in een centrale relationele database. De database is ontworpen om de kernfunctionaliteiten van het platform te ondersteunen, waaronder gebruikersbeheer, productbeheer, voorraadbeheer, bestellingen, reservaties en chatbot interacties.

2.2 Opbouw Data Model (ERD):



Figuur 2: Database ERD

De bovenstaande ERD beschrijft de structuur van de database en hoe de data onderling relateert.

Logica Data Model:

De database is logisch onderverdeeld in vier hoofddomeinen:

- **User Management Data:**
 - Bevat gegevens over gebruikers van het systeem, incl. klanten, medewerkers en administrators. Dit wordt opgeslagen in de *users* tabel.
- **Product en Inventory Data:**
 - Bevat informatie over auto-onderdelen, productcategorieën en compatibiliteit met automodellen. Deze data bevinden zich in de tabellen *products*, *categories*, *auto_model* en *product_compatibility*.
- **Transaction Data:**
 - Bevat informatie over reservaties en bestellingen die door gebruikers worden geplaatst. Deze worden opgeslagen in de tabellen *reservations*, *orders* en *order_items*.
- **Conversational Data:**
 - Bevat informatie van interacties tussen gebruikers en chatbot. Dit wordt opgeslagen in de tabellen *chats*, *messages* en *chat_summaries*.

Deze scheiding van data-domeinen zorgt voor een duidelijke structuur, betere schaalbaarheid, efficiëntie en eenvoudiger onderhoud van de database.

De belangrijkste relaties tussen entiteiten zijn:

- Eén **category** kan meerdere **products** bevatten
- Eén **product** kan compatibel zijn met meerdere **auto models**
- Eén **order** kan meerdere **order_items** bevatten
- Eén **chat** kan meerdere **messages** bevatten

Deze structuur zorgt voor normalisatie van data en voorkomt redundante opslag.

2.3 Secure Data Quality:

Performance:

Om de prestaties van databasequery's te verbeteren, zijn verschillende indexen gedefinieerd op kolommen die frequent worden gebruikt in zoekopdrachten en joins.

De volgende indexen zijn geïmplementeerd:

- **idx_product_category:** voor snellere filtering van producten per categorie.
- **idx_reservation_user:** voor efficiënte retrieval van reservaties per gebruiker.
- **idx_product_auto_comp:** om compatibele auto's voor een product sneller op te halen.

- **idx_auto_product_comp:** om compatibele producten voor een auto sneller te vinden.

Door deze indexen kan de database sneller records vinden zonder volledige tabelscans uit te voeren, wat de responstijd van de applicatie aanzienlijk verbetert.

Integrity:

Data-integriteit wordt gewaarborgd door middel van verschillende database constraints.

- **Primary Keys:** elke tabel bevat een primary key die elke record uniek identificeert.
- **Foreign Keys:** zorgt ervoor dat records alleen verwijzen naar bestaande gegevens.
- **Unique Constraints:** om dubbele identificatie te voorkomen.
- **Not Null Constraints:** velden die niet leeg mogen zijn, vb. verkoop_prijs.
- **Enumerations:** voor bepaalde velden worden vaste waarden gebruikt.

Deze constraints zorgen ervoor dat alleen geldige en consistente data in de database wordt opgeslagen.

Data security:

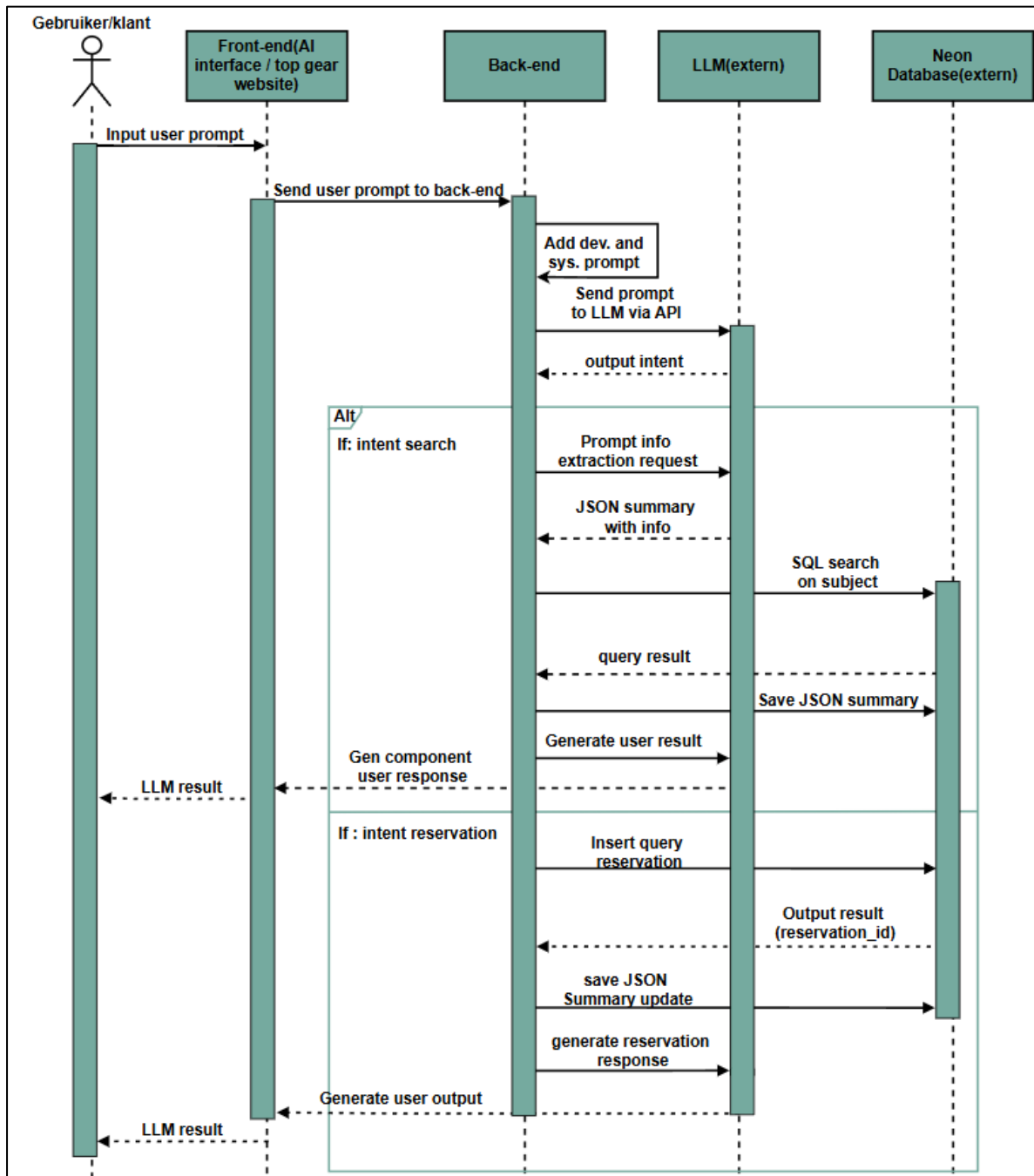
De database bevat verschillende maatregelen om de veiligheid van data te waarborgen.

- **User Roles:** systeem maakt gebruik van rol *gebaseerde* toegang tot informatie voor gebruikers.
- **Foreign Key Integrity:** dit voorkomt dat verwijzingen naar niet-bestaande records worden opgeslagen.
- **Chat Data Storage:** chatgegevens worden opgeslagen per gebruiker en per sessie, waardoor interacties traceerbaar blijven binnen het systeem.

Zowel de backend als front-end bevatten securityprotocollen wat samen de data klaar voor distributie en gebruik maakt. Deze protocollen zijn te vinden in het 'Veiligheid & privacy analyse' verslag.

2.4 Data Flow

Dit sequence diagram toont de interactiestappen tussen de gebruiker, frontend, backend, het externe LLM en de database bij het verwerken van een gebruikersvraag binnen de applicatie.



Figuur 3: Sequence Diagram

3. AI & LLM Integration

Binnen de applicatie wordt een Large Language Model (LLM) geïntegreerd om natuurlijke taalinput van gebruikers te interpreteren en te koppelen aan backendfunctionaliteit. De LLM fungeert als een interpretatie- en beslissingslaag tussen de gebruikersinterface en de applicatielogica. Hierdoor kan vrije tekst van gebruikers worden omgezet naar gestructureerde acties zoals het uitvoeren van een productzoekopdracht of het aanmaken van een reservering.

3.1 Keuze van het LLM Platform (Groq):

Voor implementatie van het LLM is gekozen voor **Groq**, een platform dat zeer snelle inference aanbiedt voor grote taalmodellen. Groq is geoptimaliseerd voor lage latency en hoge throughput, waardoor het geschikt is voor interactieve toepassingen zoals chatbots.

Binnen het systeem wordt gebruik gemaakt van het model: **Llama-3.3-70B-Versatile** wat terug te vinden is met haar implementatie technieken bij de operaties.



Het model ondersteunt onder andere:

- Natuurlijke taalinterpretatie
- Instructie-gebaseerde prompts
- Gestructureerde output (bijvoorbeeld JSON)
- Context gebaseerde interacties

De keuze voor dit model is gemaakt omdat het een goede balans biedt tussen **taalbegrip**, **outputkwaliteit en inference-snelheid**, wat belangrijk is voor real-time chatbotinteracties.

3.2 Promptstructuur (System, Developer en User Prompt):

De interactie met het LLM is opgebouwd uit een gelaagde promptstructuur. Deze structuur zorgt ervoor dat het model consistente en programmeerbare output genereert die door de backend kan worden verwerkt.

System Prompt:

De system prompt definieert de rol van het model binnen de applicatie. Hier wordt vastgelegd dat het model functioneert als een intent-classifier voor een autoparts chatbot. Daarnaast bevat deze prompt de volledige configuratie van de intent-classificatie, inclusief instructies, intentbeschrijvingen en detectieregels.

```
function buildIntentPrompt(body: ChatRequestBody) {
  const system = `
Je bent een intent-classifier voor een autopart shop chatbot.
Werk strikt volgens deze JSON configuratie.
Geef ALLEEN geldige JSON terug, zonder uitleg.
```

Figuur 4: Vb. System Prompt

Developer Configuratie:

De developerlaag bevat een gestructureerde configuratie (INTENT_CONFIG) waarin de logica voor intent-detectie is vastgelegd. Deze configuratie definieert:

- De beschikbare intents (SEARCH, RESERVATION, GENERAL, SMALLTALK)
- Beschrijvingen van elke intent
- Voorbeeldzinnen voor herkenning
- Detectieregels voor interpretatie
- Prioriteitsregels tussen intents
- Het JSON-outputschema

Door deze logica expliciet in een configuratieobject te plaatsen blijft de intent-detectie consistent en eenvoudig aanpasbaar.

User Prompt:

De user prompt bevat de daadwerkelijke gebruikersinput en aanvullende context die nodig is voor interpretatie. Deze prompt bevat:

- De huidige gebruikersvraag <User message>
- De chat summary (context van eerdere interacties) <Chat Summary>
- De recente chatgeschiedenis <Recent history>

Deze context helpt het model om korte of incomplete antwoorden correct te interpreteren binnen een lopende conversatie.

```
const user = `
User message:
${(body.message || "").trim()}

Chat summary:
${(body.chatSummary || "(none)")}

Recent history:
${(body.history || "(none)")}

Geef nu alleen JSON terug in exact dit formaat:
{
  "intent": "SEARCH",
  "confidence": 0.0
}
`.trim();
```

Figuur 5: Vb. User Prompt

3.3 JSON-gebaseerde output:

Een belangrijk onderdeel dat wij terugzien van de *system prompt* is de integratie is het gebruik van **gestructureerde JSON-output**. In plaats van alleen tekst te genereren, wordt het model geïnstrueerd om informatie terug te geven in een gestructureerd formaat.

Een voorbeeld van zo'n JSON-structuur is binnen ons systeem:

```
{
  "intent": "INFORMATIE",
  "searchState": {
    "part": "",
    "brand": "",
    "autoBrand": "Toyota",
    "model": "Supra",
    "year": "2025",
    "missingFields": [],
    "readyForDbSearch": false,
    "lastAskedField": ""
  },
  "searchResults": {
    "hasSearched": false,
    "totalFound": 0,
    "items": [],
    "lastQueryText": ""
  },
  "reservationState": {
    "part": "Front Brake Pads",
    "quantity": "1",
    "pickupDate": "",
    "selectedProductId": 33,
    "selectedProductName": "Front Brake Pads",
    "status": "pending",
    "reservationId": null,
    "expiresAt": "13 maart 2026 om 03:40",
    "lastAction": "reservering gemaakt"
  },
  "notes": [],
  "openQuestion": "spark plug"
}
```

Figuur 6: Vb. JSON output

Door gebruik te maken van JSON-output kan de backend deze informatie direct verwerken zonder extra natuurlijke taalinterpretatie. Dit maakt de integratie betrouwbaarder en vermindert de kans op interpretatiefouten.

Opm.: Als je binnen dezelfde chat besluit neemt om over een ander onderdeel of automodel te praten, dan verandert de JSON-content ook, door een nieuwe op te slaan van de opgegeven prompt.

3.4 Function calling:

Na de achterhaling van de gebruikersinput wordt op basis van de intent een specifieke backend functie aangeroepen.

Voorbeelden van deze functies zijn:

- **Search handler** – voert een productzoekopdracht uit in de database op verkregen informatie.
- **Reservation handler** – maakt een reservering aan in de database voor gebruiker.
- **General handler** – Eenvoudige conversatie over auto/ auto onderdelen.

Deze architectuur maakt gebruik van function-based routing waarbij iedere intent wordt gekoppeld aan een specifieke handler binnen de backend.

4. Infrastructuur & deployment

Deployment pipeline

1. Lokaal ontwikkelen en testen

- Zorg dat Node.js 16 en npm zijn geïnstalleerd.
- Clone repository van GitHub naar je lokale omgeving met VSCode:
<https://github.com/Shwek07/autobot.git>
- Installeer de benodigde dependencies via **npm install**
- Maak een **.env** bestand aan en voeg de databasegegevens van Neon toe (DATABASE_URL=)
- Start de development server met **npm run dev** en controleer via "<http://localhost:3000>" dat de applicatie correct werkt en verbinding maakt met de database.

2. Versiebeheer via GitHub

- Nieuwe functionaliteiten worden ontwikkeld in aparte branches om de stabiele productieversie niet te beïnvloeden.
- Zodra een feature is getest en goedgekeurd, wordt de code gemerged naar de masterbranch.

3. Automatische deployment via Vercel (CI/CD)

- Vercel detecteert wijzigingen in de masterbranch en start automatisch een buildproces.
- Tijdens dit proces worden de frontend en backend van de Next.js applicatie opgebouwd en klaargemaakt voor productie.
- Vercel zorgt voor Zero-downtime deployment, zodat de huidige versie online blijft totdat de nieuwe versie volledig klaar is.

4. Monitoring en rollback

- Na deploy wordt gecontroleerd of alle functionaliteiten correct werken.
- Indien nodig kan een eerdere stabiele versie via Vercel worden teruggezet, waardoor de impact op gebruikers minimaal is.

Server specificaties: applicatieomgeving

De applicatie draait volledig in de cloud en maakt gebruik van een serverless architectuur. De hosting en deployment worden verzorgd door *Vercel*, waardoor er geen eigen servers beheerd hoeven te worden. Vercel zorgt automatisch voor:

- Automatische scaling bij toenemend gebruik
- Zero-downtime deployments
- Beveiliging via HTTPS met automatisch SSL-certificaat

De backend wordt beheerd via Next.js API-routes, die communiceren met de Neon serverless PostgreSQL-database. Neon zorgt voor een schaalbare databaseomgeving die automatisch meegroeit met de applicatie.

De ontwikkelomgeving draait lokaal op een computer met Node.js 16, npm, VSCode. Hier wordt de applicatie getest voordat deze via GitHub en *Vercel* naar de productieomgeving wordt gedeployed.

Alle gevoelige gegevens, zoals database-credentials en API-sleutels worden opgeslagen in environment variables (.env lokaal) om veiligheid te garanderen.

5. Onderhoud en betrouwbaarheid

Om de betrouwbaarheid en veiligheid van de applicatie te waarborgen, zijn logging en monitoring essentieel. Ze zorgen ervoor dat fouten, prestatieproblemen en beveiligingsincidenten snel worden opgespoord en opgelost.

5.1 Logging

Logging wordt gebruikt om belangrijke gebeurtenissen binnen de applicatie systematisch te registreren, zodat fouten kunnen worden opgespoord en opgelost.

Wat wordt gelogd

Binnen de applicatie worden verschillende soorten gebeurtenissen geregistreerd:

- **Build- en deploymentlogs via Vercel**
Deze logs bevatten informatie over het buildproces en het deployen van nieuwe versies van de applicatie
- **Runtime errors van server-side code**
Deze worden geregistreerd in de logs van Vercel wanneer er fouten optreden tijdens het uitvoeren van serverless functies
- **Database-activiteiten en fouten via het dashboard van Neon**
Hierbij gaat het om databasequery's, verbindingsproblemen en prestatiegegevens
- **Authenticatie- en security-events via NextAuth**
Dit omvat inlogpogingen, succesvolle logins en authenticatiefouten.

Waar worden logs opgeslagen

De logs worden op verschillende plaatsen opgeslagen, afhankelijk van het type gebeurtenis:

- **Vercel Dashboard**
Opslag van buildlogs, deploymentlogs en runtimelogs van serverless functies
- **Neon Dashboard**
Overzicht van databaseprestaties, query-activiteit en foutmeldingen
- **Lokaal in Visual Studio Code**
Tijdens development worden console-logs weergegeven in de terminal of debugconsole.

5.2 Monitoring

Monitoring wordt gebruikt om continu de werking en prestaties van de applicatie te controleren. Hierdoor kunnen problemen vroegtijdig worden ontdekt.

Wat wordt gemonitord

Bij monitoring wordt onder ander gekeken naar:

- **Beschikbaarheid van de applicatie (uptime)**

Controle of de website online en bereikbaar blijft voor gebruikers

- **Het succes of falen van deployments**

Controle of nieuwe versies van de applicatie correct worden gedeployed

- **De responstijden van API's en databaseverbindingen**

Hiermee kan worden vastgesteld of onderdelen van de applicatie traag reageren

- **Ongebruikelijke of verdachte inlogpogingen**

Dit helpt bij het detecteren van mogelijke beveiligingsproblemen

Hoe te bekijken:

Voor het monitoren van de applicatie worden verschillende dashboards gebruikt

- **Vercel Dashboard**

Realtime monitoring van builds, runtime errors en function logs

- **Neon Dashboard**

Databaseprestaties, query-activiteit en foutmeldingen

Werkwijze bij fouten:

1. Fout wordt automatisch geregistreerd in Vercel of Neon
2. Ontwikkelaars analyseren de logs en achterhalen de oorzaak
3. Correcties worden doorgevoerd en lokaal getest
4. Nieuwe build wordt via Vercel gepubliceerd
5. Indien nodig: rollback naar eerdere stabiele versie

6. Schaalbaarheid van het systeem

Bij de ontwikkeling van het systeem is rekening gehouden met schaalbaarheid, zodat de applicatie kan meegroeien wanneer het aantal gebruikers toeneemt. De applicatie wordt ontwikkeld met **Visual Studio Code**, wat samenwerking tussen meerdere ontwikkelaars mogelijk maakt via versiebeheer zoals Git. Hierdoor kunnen meerdere ontwikkelaars gelijktijdig aan het project werken en kan de applicatie later eenvoudig worden uitgebreid en onderhouden.

De applicatie wordt gedeployed via **Vercel**, een cloudplatform dat gebruikmaakt van serverless hosting. Hierdoor kan de applicatie automatisch opschalen wanneer het aantal gebruikers of verzoeken toeneemt. Binnen de gratis tier zijn de beschikbare resources beperkt, waardoor het systeem geschikt is voor een kleinere groep gebruikers tijdens de ontwikkel- en testfase. Wanneer het aantal gebruikers groeit, kan het project worden opgeschaald naar een betaalde tier waarbij extra servercapaciteit beschikbaar wordt gesteld.

Voor de opslag van gegevens wordt gebruikgemaakt van **Neon Database**, een serverless PostgreSQL-database. Neon biedt eveneens een gratis tier met beperkte opslag en verwerkingscapaciteit. Wanneer de applicatie meer data moet verwerken of wanneer het aantal databaseverzoeken toeneemt, kan de database worden opgeschaald naar een hogere configuratie met meer opslag en rekenkracht.

Door deze infrastructuur kan het systeem starten met lage kosten en later eenvoudig worden opgeschaald wanneer het aantal gebruikers of de hoeveelheid data groeit.

In de toekomst kan de infrastructuur verder worden uitgebreid wanneer het systeem door een grotere organisatie of een groter aantal gebruikers wordt gebruikt.